



KOMMURI PRATAP REDDY INSTITUTE OF TECHNOLOGY

(UGC-Autonomous Institution) Ghanpur(V), Ghatkesar(M), Medchal(D), Hyderabad-500088

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING (AI&ML)

MAJOR PROJECT STAGE-1, REVIEW-2

ROLE-BASED MULTI-AGENT LLMs

for Transparent Decision-Making

A transparent decision-support system using custom Flask/Python orchestration, an OpenAI GPT API, Mock AI Mode, and four role -based agents: Advocate, Skeptic, Compliance Checker, and Adjudicator.

Under The Guidance:

Dr. Pulime Satyanarayana

PROFESSOR

DEPARTMENT OF CSE (AI&ML)

Presented by: Batch 10

N. Vishnu Kumar — 23RA1A66D4

G. Vinay — 23RA1A66D1

M. Yeshwanth — 23RA1A66E0

K. Shiva Sai — 23RA1A66A5

Abstract

This project builds a role-based multi-agent LLM system for transparent binary decisions in loan evaluation and academic admissions. A custom Flask/Python two-round debate engine coordinates four roles: Advocate presents supporting evidence, Skeptic challenges risks and weaknesses, Compliance Checker reviews policy and fairness concerns, and Adjudicator produces Approved or Rejected with a reason, explanation, and confidence score. The OpenAI GPT API is the primary backend; Mock AI Mode supports testing without an API connection. Two synthetic datasets are used: 1,000 loan records and 1,000 academic-admission records, with no real student data. JSON audit logs and SQLite store agent outputs, debate history, compliance findings, confidence scores, and final decisions.

Keywords: Multi-Agent Systems, LLMs, Explainable AI, Role-Based Debate, Bias Detection, Auditability, Loan Evaluation, Academic Admissions.

Foundations of Multi-Agent Debate

1

Du et al. (2023) — arXiv:2305.14325

Introduced multi-agent debate: several LLM instances critique each other over multiple rounds, improving factual accuracy and reasoning versus single-agent prompting.

2

Wu et al. (2024) — Multi-Agent Debate Systems

Examined role specialization and flexible conversation patterns for multi-agent LLM systems, informing this project's debate and critique design.

3

Comparative study (2024) — Debate and Adjudication

Compared collaborative agent designs, highlighting the value of role specialization, critique, adjudication, explainability, and auditability.

Project gap: This project applies these ideas through a custom Flask/Python orchestration layer and a defined two-round debate engine rather than a third-party multi-agent framework.

Bias, Fairness & Risk in Agent Systems

1

Emergent Bias and Fairness in Multi-Agent Decision Systems — [arXiv:2512.16433](#)

Studies of multi-agent debate show that agent composition can amplify or reduce bias. This project uses controlled bias probes on synthetic loan and academic-admission records by swapping demographic attributes across otherwise identical profiles. The system detects and reports potential inconsistency and bias-related outcomes; it does not guarantee fairness. The Compliance Checker provides the policy and fairness review stage.

2

Emergence of Biased Consensus in Multi-Agent LLM Debates — [arXiv:2608.02827](#)

Showed that conformity among debating agents can trigger a phase transition into collective bias, and that agent heterogeneity helps suppress it.

3

Weidinger et al. (2022) — [ACM FAccT](#)

Language-model risk research highlights discrimination and lack of accountability, motivating this project's Compliance Checker and its focus on policy review, explainability, and auditability.

Single-Model Decision Pipelines



Limitations

- Minimal explanation
- No structured challenge before finalization
- No dedicated compliance review
- Limited auditability
- No controlled demographic-attribute bias-probing step
- Difficult reproduction of decisions

A Committee of Role-Based Agents

1

Advocate

Argues in favor of the applicant / case

2

Skeptic

Challenges risks, weaknesses, and unsupported reasoning

3

Compliance-Checker

Flags policy and fairness concerns

4

Adjudicator

Synthesizes the debate into Approved or Rejected with reason, explanation, and confidence

Custom Flask/Python API and orchestration • custom two-round debate engine • OpenAI GPT API with Mock AI Mode fallback • structured JSON audit logging • SQLite storage • controlled bias probes on two synthetic datasets

Tools & Technologies Powering the System

1

Orchestration

Custom Python two-round debate engine; Flask API layer

2

Language model

OpenAI GPT API; Mock AI Mode fallback

3

Programming

Python and Flask backend implementation

4

Storage & logging

SQLite database and structured JSON audit logs

5

Datasets

1,000 synthetic loan records; 1,000 synthetic academic - admission records

6

Analytics & validation

Dashboard: decision, bias, fairness, and confidence metrics; form validation

What This Project Sets Out to Do

1

Develop a reusable role-based multi-agent decision-support system with custom Flask/Python orchestration.

2

Execute a structured two-round debate and generate Approved/Rejected decisions with reason, explanation, and confidence.

3

Record agent outputs and debate history as JSON audit logs; store cases and results in SQLite.

4

Evaluate 1,000 synthetic loan and 1,000 synthetic admission records; detect potential inconsistency and bias-related outcomes.

5

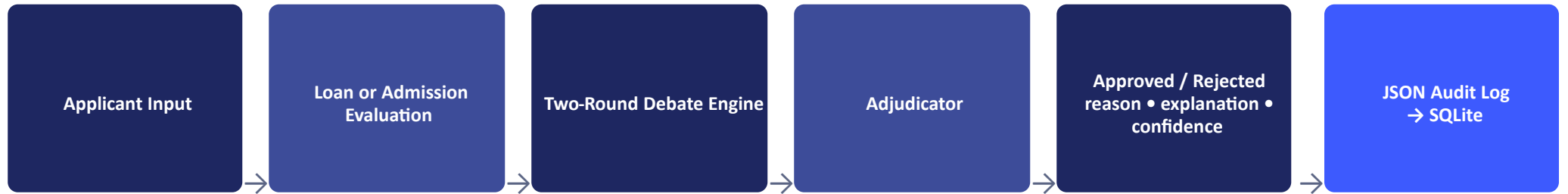
Provide dashboard analytics for decision consistency, bias alerts, fairness percentage, approval/rejection counts, and average confidence.

Architecture & Data Flow



Round 1: Advocate argues, Skeptic challenges, Compliance Checker reviews. Round 2: agents respond and Compliance Checker performs a final review before the Adjudicator decides.

Debate-Driven Decision Workflow



Round 1: Advocate argues • Skeptic challenges • Compliance Checker reviews.

Round 2: Agents respond to prior arguments and the Compliance Checker performs a final review.

The audit record stores agent outputs, debate history, compliance findings, confidence scores, and final decision explanations.

Evaluation uses two synthetic datasets: 1,000 loan records and 1,000 academic-admission records. Metrics: approval/rejection counts, Decision Consistency, Bias Alert, Fairness Percentage, Average Confidence, and agent confidence scores.

What the Project Will Deliver

1

Working Flask/Python prototype with reusable loan and admission evaluation modules

Two-round rolebased debate workflow; Approved/Rejected decisions with reasons, explanations, and confidence scores

2

Structured JSON audit logs and SQLite-backed case and result storage

Audit records capture agent outputs, debate history, compliance findings, confidence scores, and final explanations

3

Bias detection & dashboard

Potential inconsistencies; Decision Consistency, Bias Alert, Fairness Percentage, Average Confidence, approval and rejection counts

4

Reusable evaluation modules

Loan and academicadmission modules with dashboard analytics across both synthetic datasets

5

Scope note

Human review, manual override, and appeal workflow are not included in the current specification. The prototype is a decision support system focused on transparency, auditability, and bias detection.